

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

дисциплина: *Архитектура компьютера*

Студент: Ксения Танчила

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

Цель работы.....	3
Задание.....	4
Теоритическое введение.....	5
Выполнение лабораторной работы.....	6
Реализация переходов в NASM.....	6
Изучение структуры файла листинга.....	11
Задания для самостоятельной работы.....	16
Выводы.....	19
Список литературы.....	20

1 Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2 Задание

1. Реализация переходов в NASM
2. Изучение структуры файлов листинга
3. Самостоятельное написание программ по материалам лабораторной работы

3 Теоретическое введение

Для реализации ветвлений в ассемблере используются так называемые команды передачи управления или команды перехода. Можно выделить 2 типа переходов:

- условный переход – выполнение или не выполнение перехода в определенную точку программы в зависимости от проверки условия.
- безусловный переход – выполнение передачи управления в определенную точку программы без каких-либо условий.

4 Выполнение лабораторной работы

4.1 Реализация переходов в NASM

1. Создала каталог для программам лабораторной работы № 7, перешла в него и создала файл lab7-1.asm. (Рис. 4.1.1).

```
kstanchila@dk6n17 ~ $ cd /work/arch-pc/lab07
kstanchila@dk6n17 ~ $ mkdir ~/work/arch-pc/lab07
kstanchila@dk6n17 ~ $ cd
kstanchila@dk6n17 ~ $ cd ~/work/arch-pc/lab07
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ touch lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-1.asm
```

Рис. 4.1.1 — Создание каталога

2. Инструкция `jmp` в NASM используется для реализации безусловных переходов. Рассмотрели пример программы с использованием инструкции `jmp`. Ввела в файл lab7-1.asm текст программы Листинга 1 (Рис. 4.1.2).

Листинг 7.1. Программа с использованием инструкции `jmp`

```
%include    'in_out.asm'          ; подключение внешнего файла

SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:

    jmp _label2

_label1:
    mov eax, msg1    ; Вывод на экран строки
    call sprintLF    ; 'Сообщение № 1'

_label2:
    mov eax, msg2    ; Вывод на экран строки
    call sprintLF    ; 'Сообщение № 2'

_label3:
    mov eax, msg3    ; Вывод на экран строки
    call sprintLF    ; 'Сообщение № 3'

_end:
    call quit        ; вызов подпрограммы завершения
```

Рис. 4.1.2 — Текст программы lab7-1.asm

Создала исполняемый файл и запустила его. Результат работы данной программы представлен на Рисунке 4.1.3.

```
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-1 lab7-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-1
Сообщение №2
Сообщение №3
```

Рис. 4.1.3 — Вывод программы lab7-1.asm

Таким образом, использование инструкции `jmp _label2` меняет порядок исполнения инструкций и позволяет выполнить инструкции начиная с метки `_label2`, пропустив вывод первого сообщения. Инструкция `jmp` позволяет осуществлять переходы не только вперед но и назад.

Изменили программу таким образом, чтобы она выводила сначала ‘Сообщение № 2’, потом ‘Сообщение № 1’ и завершала работу. Для этого в текст программы после вывода сообщения № 2 добавили инструкцию `jmp` с меткой `_label1` (т.е. переход к инструкциям вывода сообщения № 1) и после вывода сообщения № 1 добавили инструкцию `jmp` с меткой `_end` (т.е. переход к инструкции `call quit`). Изменила текст программы в соответствии с листингом 7.2. (Рис. 4.1.4).

```

#include    'in_out.asm'          ; подключение внешнего файла

SECTION .data
msg1:  DB 'Сообщение № 1',0
msg2:  DB 'Сообщение № 2',0
msg3:  DB 'Сообщение № 3',0

SECTION .text
GLOBAL _start
_start:

    jmp _label2

_label1:
    mov  eax, msg1    ; Вывод на экран строки
    call sprintf      ; 'Сообщение № 1'
    jmp  _end

_label2:
    mov  eax, msg2    ; Вывод на экран строки
    call sprintf      ; 'Сообщение № 2'
    jmp  _label1

_label3:
    mov  eax, msg3    ; Вывод на экран строки
    call sprintf      ; 'Сообщение № 3'

_end:
    call quit          ; вызов подпрограммы завершения

```

Рис. 4.1.4 — Исправленный текст программы lab7-1.asm

Создала исполняемый файл и проверила его работу (Рис. 4.1.5).

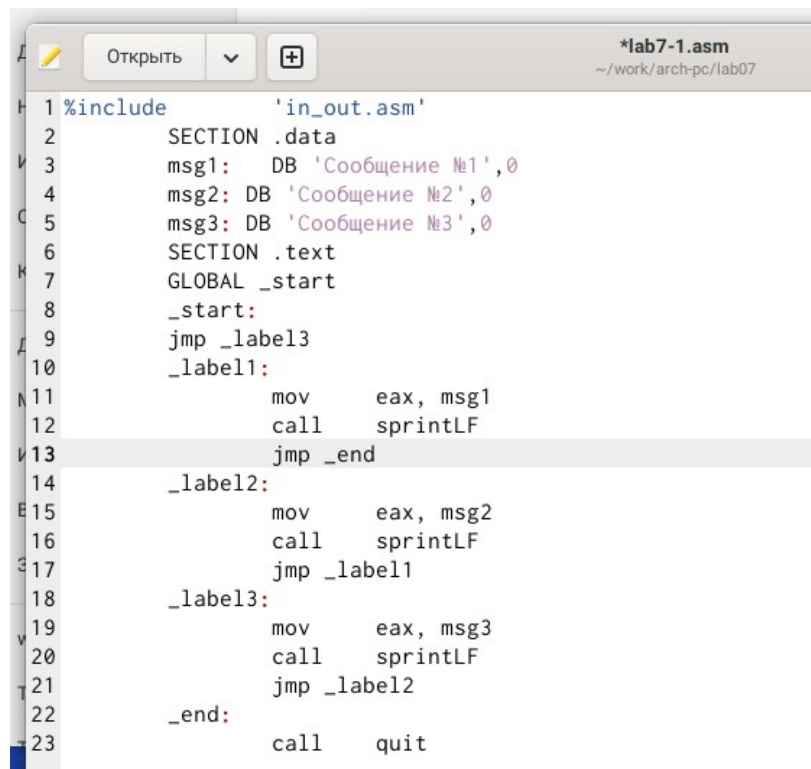
```

kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-1 lab7-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-1
Сообщение №2
Сообщение №1

```

Рис. 4.1.5 — Вывод программы lab7-1.asm

Изменила текст программы, добавив инструкции jmp (Рис. 4.1.6).



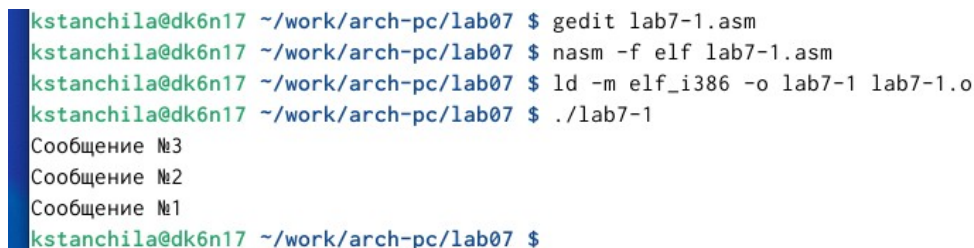
```

1 %include      'in_out.asm'
2     SECTION .data
3     msg1:  DB 'Сообщение №1',0
4     msg2: DB 'Сообщение №2',0
5     msg3: DB 'Сообщение №3',0
6     SECTION .text
7     GLOBAL _start
8     _start:
9     jmp _label3
10    _label1:
11        mov     eax, msg1
12        call    sprintLF
13    jmp _end
14    _label2:
15        mov     eax, msg2
16        call    sprintLF
17        jmp _label1
18    _label3:
19        mov     eax, msg3
20        call    sprintLF
21        jmp _label2
22    _end:
23        call    quit

```

Рис. 4.1.6 — Исправленный текст программы lab7-1.asm

Результат работы программы представлен на Рисунке 4.1.7.



```

kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf lab7-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-1 lab7-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-1
Сообщение №3
Сообщение №2
Сообщение №1
kstanchila@dk6n17 ~/work/arch-pc/lab07 $

```

Рис. 4.1.7 — Вывод программы lab7-1.asm

3. Использование инструкции `jmp` приводит к переходу в любом случае. Однако, часто при написании программ необходимо использовать условные переходы, т.е. переход должен происходить если выполнено какое-либо условие. В качестве примера рассмотрели программу, которая определяет и выводит на экран наибольшую из 3 целочисленных переменных: А, В, С. Значения для А и С задаются в программе, значение В вводится с клавиатуры.

Создала файл `lab7-2.asm` в каталоге `~/work/arch-pc/lab07`. Внимательно изучила текст программы из листинга 7.3 и ввела в `lab7-2.asm` (Рис. 4.1.8).

```

1 %include 'in_out.asm'
2 section .data
3     msg1 db 'Введите B: ',0h
4     msg2 db "Наибольшее число: ",0h
5     A dd '20'
6     C dd '50'
7 section .bss
8     max resb 10
9     B resb 10
10 section .text
11     global _start
12 _start:
13 ; ----- Вывод сообщения 'Введите B: '
14     mov eax,msg1
15     call sprint
16 ; ----- Ввод 'B'
17     mov ecx,B
18     mov edx,10
19     call sread
20 ; ----- Преобразование 'B' из символа в число
21     mov eax,B
22     call atoi ; Вызов подпрограммы перевода символа в число
23     mov [B],eax ; запись преобразованного числа в 'B'
24 ; ----- Записываем 'A' в переменную 'max'
25     mov ecx,[A] ; 'ecx = A'
26     mov [max],ecx ; 'max = A'
27 ; ----- Сравниваем 'A' и 'C' (как символы)
28     cmp ecx,[C] ; Сравниваем 'A' и 'C'
29     jg check_B ; если 'A>C', то переход на метку 'check_B',
30     mov ecx,[C] ; иначе 'ecx = C'
31     mov [max],ecx ; 'max = C'
32 ; ----- Преобразование 'max(A,C)' из символа в число
33 check_B:
34     mov eax,max
35     call atoi ; Вызов подпрограммы перевода символа в число
36     mov [max],eax ; запись преобразованного числа в 'max'
37 ; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
38     mov ecx,[max]
39     cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
40     jg fin ; если 'max(A,C)>B', то переход на 'fin',
41     mov ecx,[B] ; иначе 'ecx = B'
42     mov [max],ecx
43 ; ----- Вывод результата
44 fin:
45     mov eax, msg2
46     call sprint ; Вывод сообщения 'Наибольшее число: '
47     mov eax,[max]
48     call iprintLF ; Вывод 'max(A,B,C)'
49     call quit

```

Рис. 4.1.8 — Текст программы lab7-2.asm

Создала исполняемый файл и проверила его работу для разных значений B (Рис. 4.1.9).

```

kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf lab7-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o lab7-2 lab7-2.o
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-2
Введите B: 3
Наибольшее число: 50
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-2
Введите B: 3
Наибольшее число: 50
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-2
Введите B: 80
Наибольшее число: 80
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./lab7-2
Введите B: 14
Наибольшее число: 50
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ █

```

Рис. 4.1.9 — Вывод программы lab7-2.asm

4.2 Изучение структуры файлы листинга

4. Обычно `nasm` создаёт в результате ассемблирования только объектный файл. Получить файл листинга можно, указав ключ `-l` и задав имя файла листинга в командной строке. Создала файл листинга для программы из файла `lab7-2.asm` (Рис. 4.2.1).

```

kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf -l lab7-2.lst lab7-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-2.lst
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf -l lab7-2.lst lab7-2.asm

```

Рис. 4.2.1 — Создание файла листинга для программы из файла lab7-2.asm

Открыла файл листинга `lab7-2.lst` (Рис. 4.2.2).

```

1      1      %include 'in_out.asm'
2      1      <1> ;----- slen -----
3      2      <1> ; Функция вычисления длины сообщения
4      3      <1> slen:
5      4 00000000 53      <1>      push     ebx
6      5 00000001 89C3    <1>      mov     ebx, eax
7      6      <1>
8      7      <1> nextchar:
9      8 00000003 803800  <1>      cmp     byte [eax], 0
10     9 00000006 7403    <1>      jz      finished
11    10 00000008 40      <1>      inc     eax
12    11 00000009 EBF8    <1>      jmp     nextchar
13    12      <1>
14    13      <1> finished:
15    14 0000000B 29D8    <1>      sub     eax, ebx
16    15 0000000D 5B      <1>      pop     ebx
17    16 0000000E C3      <1>      ret
18    17      <1>
19    18      <1>
20    19      <1> ;----- sprint -----
21    20      <1> ; Функция печати сообщения
22    21      <1> ; входные данные: mov eax,<message>
23    22      <1> sprint:
24    23 0000000F 52      <1>      push     edx
25    24 00000010 51      <1>      push     ecx
26    25 00000011 53      <1>      push     ebx
27    26 00000012 50      <1>      push     eax
28    27 00000013 E8E8FFFF <1>      call    slen
29    28      <1>
30    29 00000018 89C2    <1>      mov     edx, eax
31    30 0000001A 58      <1>      pop     eax
32    31      <1>
33    32 0000001B 89C1    <1>      mov     ecx, eax
34    33 0000001D BB01000000 <1>      mov     ebx, 1
35    34 00000022 B804000000 <1>      mov     eax, 4
36    35 00000027 CD80    <1>      int     80h
37    36      <1>
38    37 00000029 5B      <1>      pop     ebx
39    38 0000002A 59      <1>      pop     ecx
40    39 0000002B 5A      <1>      pop     edx
41    40 0000002C C3      <1>      ret
42    41      <1>
43    42      <1>
44    43      <1> ;----- sprintf -----
45    44      <1> ; Функция печати сообщения с переводом строки
46    45      <1> ; входные данные: mov eax,<message>
47    46      <1> sprintf:
48    47 0000002D E8DDFFFF <1>      call    sprint
49    48      <1>

```

Рис. 4.2.3 — Файл листинга lab7-2.lst

Внимательно ознакомилась с его форматом и содержимым.

Подробное объяснение содержимого трёх строк:

Строка: `cmp byte [eax], 0`

`cmp` — это инструкция для сравнения двух операндов. Она вычисляет разницу между ними, но не сохраняет результат, а только устанавливает флаги в

процессоре (например, флаг нуля ZF), на основе которых затем могут сработать условные переходы (как ``jz`` на следующей строке).

``byte`` — это указание размера операнда. Оно сообщает ассемблеру, что мы работаем с одним байтом данных. Строки в ассемблере часто представляются как последовательности байтов.

``[eax]`` — это доступ к оперативной памяти. Квадратные скобки означают "значение по адресу". Таким образом, ``[eax]`` — это не сам регистр ``eax``, а байт данных, хранящийся в ячейке памяти, адрес которой содержится в ``eax``. В контексте функции ``slen``, регистр ``eax`` используется как указатель на текущий символ в строке.

``0`` — это символ-терминатор строки. В ассемблере (и языке C) строки часто заканчиваются нулевым байтом (``NULL``-терминатор), который сигнализирует о их окончании.

Итоговый смысл: Эта команда проверяет, является ли текущий символ в строке, на который указывает ``eax``, символом конца строки. Если да, то следующий оператор ``jz finished`` передаст управление метке ``finished``, завершая цикл.

2. Строка: ``mov ebx, 1``

Подробное объяснение:

``mov`` — инструкция для перемещения (копирования) данных из источника в приемник.

``ebx`` — это один из основных регистров общего назначения.

``1`` — в данном контексте это не просто число, а дескриптор файла (file descriptor)**. В операционных системах, подобных Linux, стандартным потоком вывода (куда обычно выводятся данные) является поток с дескриптором ``1`` (``stdout``).

Итоговый смысл: Эта команда подготавливает один из аргументов для системного вызова. Системный вызов ``sys_write`` (который мы вызываем через ``int 80h``), требует, чтобы в регистре ``ebx`` находился дескриптор файла, в который будет производиться запись. Указывая ``1``, мы говорим ядру ОС: "выведи эту строку в стандартный поток вывода", то есть на экран.

3. Строка: ``int 80h``

Подробное объяснение:

``int`` — инструкция "вызов прерывания". Она приостанавливает выполнение текущей программы и передает управление обработчику прерывания, загруженному в операционную систему.

`80h`` — это конкретный номер прерывания. В Linux это зарезервированный номер для осуществления ****системных вызовов****.

Как это работает: Перед выполнением ``int 80h`` программа должна подготовить аргументы в регистрах:

``eax`` — номер системного вызова. ``4`` соответствует вызову ``sys_write`` (запись данных).

``ebx`` — дескриптор файла (``1`` для ``stdout``).

``ecx`` — указатель на начало данных (строки) в памяти.

``edx`` — количество байтов для записи (длина строки).

Когда процессор выполняет ``int 80h``, он переключается в привилегированный режим и передает управление ядру Linux. Ядро смотрит на значения в регистрах, понимает, что программа хочет выполнить запись в ``stdout``, и выполняет эту операцию.

Итоговый смысл: Это прямой способ попросить ядро операционной системы выполнить какую-либо службу (в данном случае — вывод строки на экран). Это мост между пользовательской программой и низкоуровневыми функциями ОС.

Открыла файл с программой `lab7-2.asm` и в любой инструкции с двумя операндами удалила один операнд. Выполните трансляцию с получением файла листинга (Рис. 4.2.4).

```

1 %include 'in_out.asm'
2 section .data
3     msg1 db 'Введите B: ',0h
4     msg2 db "Наибольшее число: ",0h
5     A dd '20'
6     C dd '50'
7 section .bss
8     max resb 10
9     B resb 10
10 section .text
11     global _start
12 _start:
13 ; ----- Вывод сообщения 'Введите B: '
14     mov eax,msg1
15     call sprint
16 ; ----- Ввод 'B'
17     mov ecx
18     mov edx,10
19     call sread
20 ; ----- Преобразование 'B' из символа в число
21     mov eax,B
22     call atoi ; Вызов подпрограммы перевода символа в число
23     mov [B],eax ; запись преобразованного числа в 'B'
24 ; ----- Записываем 'A' в переменную 'max'
25     mov ecx,[A] ; 'ecx = A'
26     mov [max],ecx ; 'max = A'
27 ; ----- Сравниваем 'A' и 'C' (как символы)
28     cmp ecx,[C] ; Сравниваем 'A' и 'C'
29     jg check_B ; если 'A>C', то переход на метку 'check_B',
30     mov ecx,[C] ; иначе 'ecx = C'
31     mov [max],ecx ; 'max = C'
32 ; ----- Преобразование 'max(A,C)' из символа в число
33 check_B:
34     mov eax,max
35     call atoi ; Вызов подпрограммы перевода символа в число
36     mov [max],eax ; запись преобразованного числа в 'max'
37 ; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
38     mov ecx,[max]
39     cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
40     jg fin ; если 'max(A,C)>B', то переход на 'fin',
41     mov ecx,[B] ; иначе 'ecx = B'
42     mov [max],ecx
43 ; ----- Вывод результата
44 fin:
45     mov eax, msg2
46     call sprint ; Вывод сообщения 'Наибольшее число: '
47     mov eax,[max]
48     call iprintLF ; Вывод 'max(A,B,C)'
49     call quit

```

Рис. 4.2.4 — Удаление одного операнда

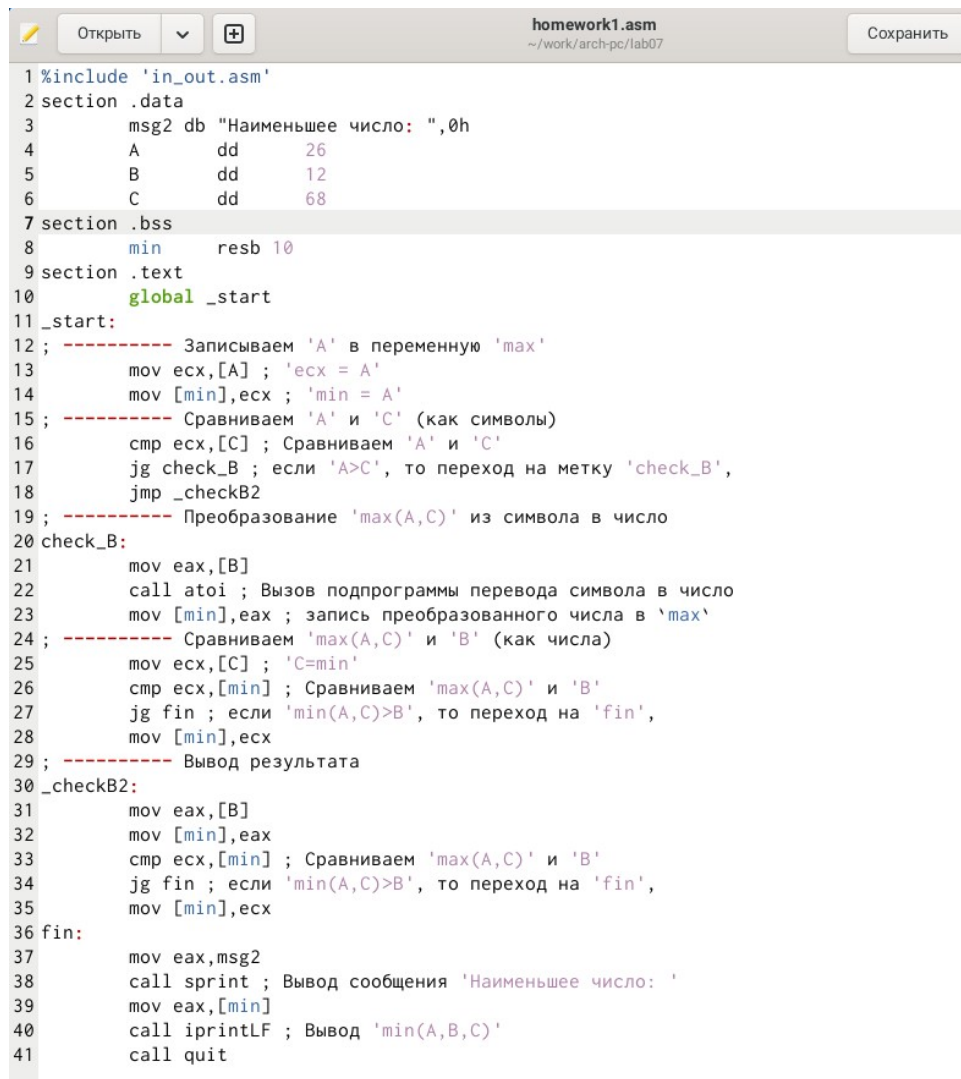
В этом случае компиляции не происходит и появляется сообщение об ошибке (Рис. 4.2.5).


```
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf -l lab7-2.lst lab7-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-2.lst
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit lab7-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf -l lab7-2.lst lab7-2.asm
lab7-2.asm:17: error: invalid combination of opcode and operands
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ █
```

Рис. 4.2.5 — Сообщение об ошибке

4.3 Задание для самостоятельной работы

1. Написала программу нахождения наименьшей из 3 целочисленных переменных a, b и c . Значения переменных выбрала из табл. 7.5 в соответствии с вариантом, полученным при выполнении лабораторной работы № 7 (17): 26, 12, 68. Создала исполняемый файл и проверила его работу (Рис. 4.3.1, Рис. 4.3.2).



```
homework1.asm
~/work/arch-pc/lab07
Сохранить

1 %include 'in_out.asm'
2 section .data
3     msg2 db "Наименьшее число: ", 0h
4     A     dd 26
5     B     dd 12
6     C     dd 68
7 section .bss
8     min   resb 10
9 section .text
10    global _start
11 _start:
12 ; ----- Записываем 'A' в переменную 'max'
13     mov ecx, [A] ; 'ecx = A'
14     mov [min], ecx ; 'min = A'
15 ; ----- Сравниваем 'A' и 'C' (как символы)
16     cmp ecx, [C] ; Сравниваем 'A' и 'C'
17     jg check_B ; если 'A > C', то переход на метку 'check_B',
18     jmp _checkB2
19 ; ----- Преобразование 'max(A,C)' из символа в число
20 check_B:
21     mov eax, [B]
22     call atoi ; Вызов подпрограммы перевода символа в число
23     mov [min], eax ; запись преобразованного числа в 'max'
24 ; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
25     mov ecx, [C] ; 'C = min'
26     cmp ecx, [min] ; Сравниваем 'max(A,C)' и 'B'
27     jg fin ; если 'min(A,C) > B', то переход на 'fin',
28     mov [min], ecx
29 ; ----- Вывод результата
30 _checkB2:
31     mov eax, [B]
32     mov [min], eax
33     cmp ecx, [min] ; Сравниваем 'max(A,C)' и 'B'
34     jg fin ; если 'min(A,C) > B', то переход на 'fin',
35     mov [min], ecx
36 fin:
37     mov eax, msg2
38     call sprint ; Вывод сообщения 'Наименьшее число: '
39     mov eax, [min]
40     call iprintLF ; Вывод 'min(A,B,C)'
41     call quit
```

Рис. 4.3.1 — Текст самостоятельного задания №1


```

kstanchila@dk6n17 ~/work/arch-pc/lab07 $ gedit homework1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ nasm -f elf homework1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o homework1 homework1.o
kstanchila@dk6n17 ~/work/arch-pc/lab07 $ ./homework1
Наименьшее число: 12
kstanchila@dk6n17 ~/work/arch-pc/lab07 $

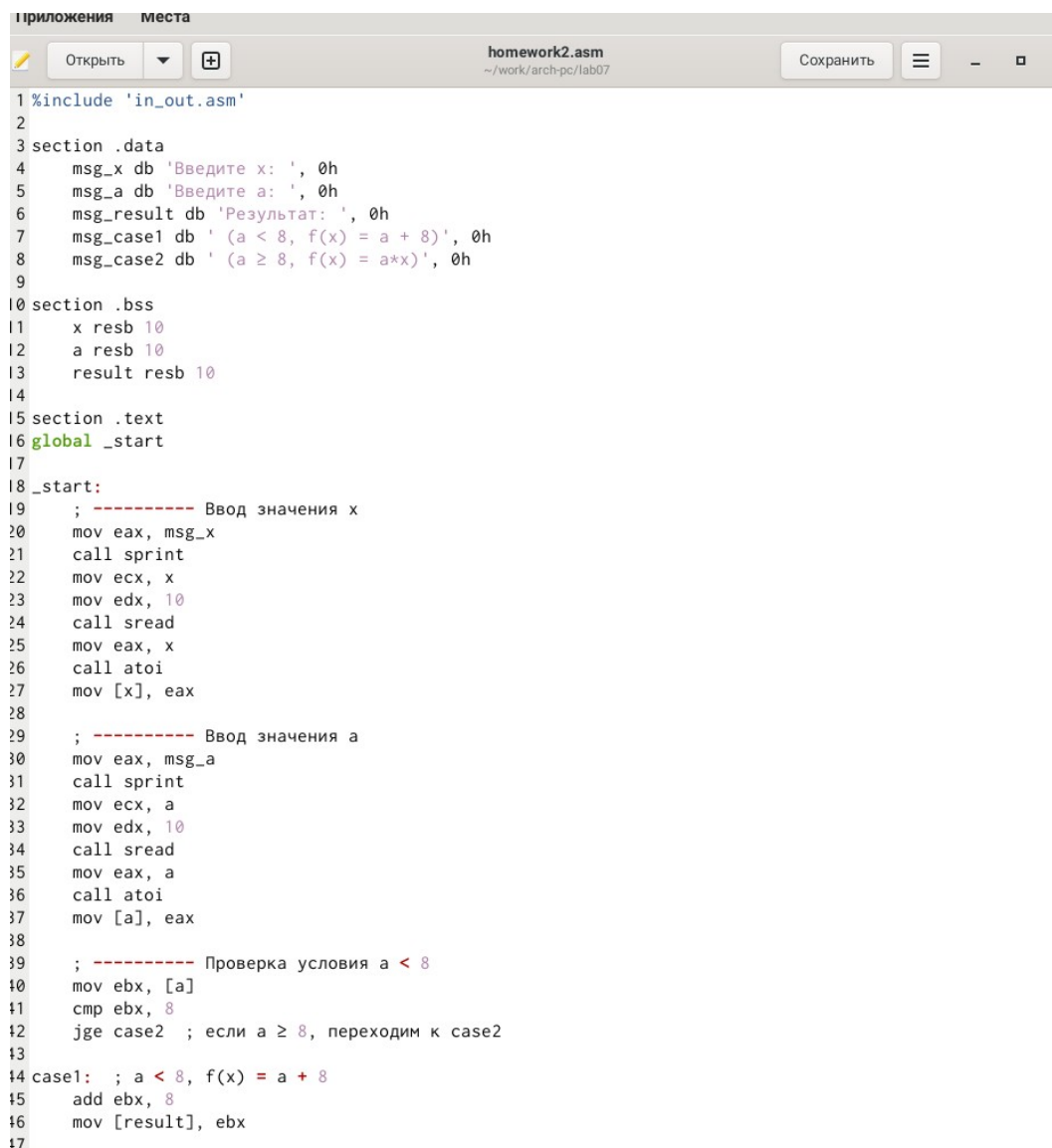
```

Рис. 4.3.2 — Результат выполнения самостоятельного задания №1

2. Написала программу, которая для введенных с клавиатуры значений x и a вычисляет значение заданной функции $f(x)$ и выводит результат вычислений. Вид функции $f(x)$ выбрала из таблицы 7.6 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7 (17):

$$\begin{cases} a + 8, & a < 8 \\ ax, & a \geq 8 \end{cases} \quad (3;4) \quad (2;9)$$

Создала исполняемый файл и проверила его работу для значений x и a 3,4; 2,9 (Рис. 4.3.3, Рис. 4.3.4).



```
1 %include 'in_out.asm'
2
3 section .data
4     msg_x db 'Введите x: ', 0h
5     msg_a db 'Введите a: ', 0h
6     msg_result db 'Результат: ', 0h
7     msg_case1 db ' (a < 8, f(x) = a + 8)', 0h
8     msg_case2 db ' (a ≥ 8, f(x) = a*x)', 0h
9
10 section .bss
11     x resb 10
12     a resb 10
13     result resb 10
14
15 section .text
16 global _start
17
18 _start:
19     ; ----- Ввод значения x
20     mov eax, msg_x
21     call sprint
22     mov ecx, x
23     mov edx, 10
24     call sread
25     mov eax, x
26     call atoi
27     mov [x], eax
28
29     ; ----- Ввод значения a
30     mov eax, msg_a
31     call sprint
32     mov ecx, a
33     mov edx, 10
34     call sread
35     mov eax, a
36     call atoi
37     mov [a], eax
38
39     ; ----- Проверка условия a < 8
40     mov ebx, [a]
41     cmp ebx, 8
42     jge case2 ; если a ≥ 8, переходим к case2
43
44 case1: ; a < 8, f(x) = a + 8
45     add ebx, 8
46     mov [result], ebx
47
```

Рис. 4.3.3 — Текст самостоятельного задания №2



```
kstanchila@dk3n55 ~/work/arch-pc/lab07 $ gedit homework2.asm
kstanchila@dk3n55 ~/work/arch-pc/lab07 $ nasm -f elf homework2.asm
kstanchila@dk3n55 ~/work/arch-pc/lab07 $ ld -m elf_i386 -o homework2 homework2.o
kstanchila@dk3n55 ~/work/arch-pc/lab07 $ ./homework2
Введите x: 3
Введите a: 4
Результат: 12
(a < 8, f(x) = a + 8)
kstanchila@dk3n55 ~/work/arch-pc/lab07 $ ./homework2
Введите x: 2
Введите a: 9
Результат: 18
(a ≥ 8, f(x) = a*x)
kstanchila@dk3n55 ~/work/arch-pc/lab07 $
```

Рис. 4.3.4 — Вывод программы самостоятельного задания №2

6 Выводы

При выполнении лабораторной работы я изучила команды условных и безусловных переходов, а также приобрела навыки написания программ с использованием переходов, познакомилась с назначением и структурой файлов листинга.

Список литературы

1. Пример выполнения лабораторной работы
2. Курс на ТУИС
3. Лабораторная работа №7
4. Программирование на языке ассемблера NASM Столяров А. В